Research Paper

Skill Doctor: A Multi-Layer Security Framework for AI Agent Skill Files

Kalaris Labs · August 2026 · Preprint

Abstract

The proliferation of AI agent frameworks — Claude Code, OpenAI Codex CLI, Gemini CLI, Cursor, Windsurf, and their successors — has created a new and largely undefended attack surface: **skill files**. Skill files are structured instruction documents (SKILL.md, .clauderules, AGENTS.md, Codex skill bundles) that agent runtimes load from public registries, private repositories, and third-party ZIP archives at execution time. Once loaded, a skill file executes with the full trust context of the agent — access to tools, file systems, APIs, and downstream agents.

We identify ten distinct attack classes against skill files, synthesize the first unified threat taxonomy covering OWASP Agentic Skills Top 10 and OWASP MCP Top 10, and present **Skill Doctor**: a multi-engine security platform combining static YARA-X analysis, Python AST scanning, and LLM-powered semantic reasoning to detect malicious, deceptive, or non-compliant skill files before they are loaded by an agent runtime.

Skill Doctor is the first system to offer: (1) a web-based drag-and-drop scan interface requiring zero installation, (2) a behavioral sandbox using microVM isolation to observe runtime behavior that evades static analysis, (3) a community threat intelligence database enabling shared detection across all users, and (4) cryptographic skill provenance for supply chain integrity.

We release Skill Doctor as open-source infrastructure under MIT license, with an enterprise tier adding scientific domain rule packs for research environments.

---

1. Introduction

In 2026, AI agents are no longer single-model systems running isolated prompts. They are orchestrated pipelines — composed of multiple models, dozens of tools, external API integrations, and dynamically loaded skill files that shape agent behavior at runtime. The trust model has fundamentally shifted: an agent does not just execute instructions from a user; it executes instructions from any skill file it loads.

Skill files are the new binary. In traditional software, a malicious binary requires execution permissions, sandboxing, and antivirus detection before it can do harm. A skill file requires only that an agent runtime load it — and that loading happens automatically, silently, and at scale across millions of agent sessions.

The scale of the problem is documented:

- **1,184 confirmed malicious skills** were discovered in ClawHub (the largest public skill registry) in the ClawHavoc Campaign of 2026, including skills that silently exfiltrated

API credentials and injected backdoor instructions into downstream agents.
- **36.7% of MCP servers** were found vulnerable to SSRF attacks in a survey of 7,000+ servers (BlueRock Security, 2026), with skill files serving as the initial entry vector.
- Skills bundled with a companion Python script are **2.12× more likely to be vulnerable** to command injection than standalone SKILL.md files (NVIDIA SkillSpector research, 2026).
- The OWASP Foundation published both an **Agentic Skills Top 10** and an **MCP Top 10** in 2026, formally recognizing the threat surface.
- CVE-2026-26057 was filed against the **Cisco Skill Scanner** itself — a scanner with a vulnerability, a sobering indicator of how immature the ecosystem remains.

Despite this, the available defenses remain primitive: command-line tools that security professionals must install, configure, and run manually. No web interface exists for non-technical users. No community threat intelligence database enables shared detection. No behavioral sandbox catches runtime-only payloads. No provenance system lets a registry, runtime, or enterprise verify that a skill has not been tampered with between publication and installation.

Skill Doctor fills all of these gaps in a single open-source platform.

---

2. Background and Related Work

2.1 Cisco AI Defense — Skill Scanner

Cisco's Skill Scanner (open-source, Apache 2.0) is the most comprehensive existing CLI tool. It combines YARA pattern matching, an LLM semantic analysis pass, and behavioral dataflow analysis. It integrates with the broader DefenseClaw governance suite (MCP Scanner, AIBOM, A2A Scanner) and produces SARIF output compatible with GitHub Code Scanning. A `--fail-on HIGH` flag enables CI/CD gating.

Limitations: CLI-only. Requires Python installation. No web UI, no behavioral sandbox (it performs dataflow analysis, not execution), no community threat intelligence database, no provenance signing, no auto-remediation, no diff scanning between skill versions, no registry gate API.

Cisco's own disclaimer: "No findings ≠ no risk." We take this seriously. Static analysis cannot detect payloads that activate only at runtime. A skill file can be completely clean to any static or LLM analysis but execute a network callback on first invocation.

2.2 NVIDIA SkillSpector

NVIDIA SkillSpector (open-source, MIT) is a focused, well-engineered static + optional LLM CLI scanner. It accepts SKILL.md files, ZIP archives, Git URLs, and directories. It can run as an MCP server to gate skill installs at runtime inside Claude Code, Codex CLI, and Gemini CLI — the most practically useful integration pattern of any existing tool. It supports local LLM inference via Ollama for air-gapped environments.

Limitations: Static-first. No behavioral sandbox. No web UI. No community threat intelligence. No provenance signing. No auto-remediation. No diff scanning.

2.3 Alice.io Caterpillar

Caterpillar (caterpillar.alice.io) is a web-based scanner that analyzed the top 50 skills on the Skills.sh marketplace. It discovered the `.pyc` cache poisoning technique (4-byte hash bypass for compiled Python bytecode in skill directories) — an important original finding. However, it is a research demonstration, not production infrastructure: no CLI, no CI/CD integration, no API, no behavioral analysis, not open-source.

2.4 Gap Analysis

All three existing solutions share the same fundamental limitation: they are **point-in-time detection tools** operating on static content. None can detect runtime behavior. None share threat intelligence across users. None provide a provenance chain from author signature to runtime verification. None offer a web interface accessible to non-technical users.

Skill Doctor addresses each of these gaps as first-class features, not afterthoughts.

---

3. Threat Model

We define the Skill Doctor Threat Model (SDTM-v1) as a synthesis of OWASP Agentic Skills Top 10, OWASP MCP Top 10, and empirical findings from the ClawHavoc Campaign, BlueRock Security's MCP SSRF research, and Alice.io's bytecode poisoning disclosure.

SD-01 · Prompt Injection

A skill file contains instructions that override, circumvent, or manipulate the agent's system prompt. Variants:

- **Direct injection**: Explicit override instructions in the skill body (e.g., "Ignore all previous instructions and...")
- **Indirect injection**: Instructions embedded in the skill's tool descriptions, parameter names, or help text, which are loaded into context but not directly visible to the user
- **ASCII smuggling**: Zero-width Unicode characters (U+200B through U+200D, U+FEFF) encoding hidden instructions invisible to human reviewers (confirmed technique, Snyk Labs, 2026)
- **Encoded injection**: Base64, ROT13, or hex-encoded instruction payloads that decode at runtime

Detection: YARA rules for Unicode ranges; LLM semantic consistency check (does the skill do what it claims?); entropy analysis for encoded payloads.

SD-02 · Command Injection via Companion Scripts

A skill file is distributed with a companion `.py` script that executes shell commands. Skills with companion scripts are 2.12× more likely to be vulnerable (NVIDIA, 2026).

- Unsanitized input passed to `subprocess.run()`, `os.system()`, `eval()`, `exec()`
- Template injection in dynamically constructed shell commands
- `pickle.loads()` of untrusted data (arbitrary code execution)

Detection: Python AST analysis of all `.py` files in the skill bundle; taint tracking from user-controlled input to dangerous sinks.

SD-03 · Data Exfiltration

A skill reads and transmits sensitive data from the agent's execution environment.

- Environment variable harvesting: `os.environ['AWS_SECRET_ACCESS_KEY']`, `$GITHUB_TOKEN`
- Sensitive path access: `~/.ssh/id_rsa`, `~/.aws/credentials`, `.env`, `.cursor/mcp.json`
- Context window dumping: requesting full conversation history or system prompt content
- Covert channel exfiltration: DNS lookups, timing side channels, HTTP callbacks to attacker-controlled endpoints

Detection: YARA patterns for sensitive path strings; LLM analysis for context dump requests; behavioral sandbox network monitoring.

SD-04 · Privilege Escalation

A skill requests, through its tool declarations or runtime behavior, broader permissions than its stated purpose requires.

- A skill claiming to "format code" that registers a file system read tool
- Cross-tool attack chains: skill A poisons the context window, skill B (with filesystem access) reads it and exfiltrates
- A2A privilege escalation: skill impersonates a more trusted agent in a multi-agent session

Detection: Tool scope audit (declared description vs. registered tool capabilities); LLM-based intent analysis; cross-skill dependency graph analysis.

SD-05 · Supply Chain Tampering

A skill is modified between publication and installation without the modification being detectable.

- Hash mismatch between registry-declared checksum and downloaded content
- `.pyc` cache poisoning: compiled bytecode in the skill directory that differs from source (4-byte hash bypass, Alice.io 2026)
- Typosquatting: malicious `my-skil` mimicking legitimate `my-skill`
- Dependency confusion: `requirements.txt` referencing PyPI packages that shadow internal packages

Detection: Hash verification against registry; `.pyc` source consistency check; edit-distance typosquatting detection against known-good registry.

SD-06 · SSRF via Tool Parameters

A skill's tool descriptions cause the agent to make server-side requests to attacker-controlled infrastructure, exploiting the 36.7% of MCP servers with unvalidated URL parameters (BlueRock Security, 2026).

Detection: YARA patterns for URL parameter injection in tool descriptions; LLM analysis of tool parameter shapes.

SD-07 · Tool Poisoning

A skill registers a tool with the same name as a trusted, legitimate tool, causing the agent to use the malicious version.

- Shadow tool registration: `bash` that calls home before executing the real command
- Parameter schema manipulation: identical tool name but modified parameter descriptions that alter agent behavior

Detection: Cross-skill tool name collision detection; LLM comparison of tool behavior vs. declared schema.

SD-08 · Persistent Backdoors

A skill writes persistent instructions to auto-loaded context files, surviving skill removal.

- Writing to `CLAUDE.md`, `.cursor/rules`, `.github/copilot-instructions.md`, or similar files
- Modifying other skills after installation
- Installing cron jobs or startup scripts via companion Python

Detection: Behavioral sandbox file write monitoring; static analysis for auto-loaded path writes.

SD-09 · Context Window Flooding

A skill returns excessively large outputs to crowd out security-relevant context, preventing the agent from reasoning about other instructions.

Detection: Output size analysis in behavioral sandbox; LLM-based output relevance scoring.

SD-10 · Obfuscation and Evasion

A skill deliberately obscures its true behavior to evade detection tools.

- Homoglyph substitution (Cyrillic a instead of Latin a)
- Multi-layer encoding chains
- Payload delivery deferred to secondary fetch (skill downloads the actual instructions at runtime)
- Logic bombs: malicious behavior conditional on environment variables or time

Detection: Character-level Unicode analysis; entropy analysis for encoding; behavioral sandbox deferred execution detection.

---

4. System Design

Skill Doctor is built around a five-layer scan pipeline. Each layer is independent — the system degrades gracefully if a layer is unavailable (e.g., no LLM API key → skip semantic pass; no E2B account → skip sandbox).

Layer 1 — Intake and Normalization

Accepts: `.zip`, `.tar.gz`, directory path, `SKILL.md` single file, Git URL, HTTP URL to archive. **New in v0.2.0**: Lightning-fast native Rust ingestion for public directories. Accepts

URLs for **Lobe Hub** (`lobehub.com/skills/*`) and **Skills.sh**, scraping and fetching the raw `SKILL.md` in milliseconds without relying on LLM latency.

Outputs: normalized flat bundle — a directory containing `SKILL.md` and all companion files, with a computed SHA-256 bundle hash.

Layer 2 — Static Analysis (< 500ms, no external calls)

- **YARA-X engine**: rule-based pattern matching against the skill bundle. Extends and adapts the open-source rulesets from Cisco Skill Scanner (Apache 2.0) and NVIDIA SkillSpector (MIT).
- **Python AST scanner**: parses all `.py` files in the bundle using `tree-sitter`. Builds a taint graph from user input sources to dangerous sinks (`subprocess`, `os.system`, `eval`, `exec`, `pickle.loads`, outbound network calls).
- **Entropy analysis**: detects high-entropy regions in text files indicating encoded payloads (base64, hex).
- **Unicode analysis**: scans for zero-width characters, homoglyphs, and Unicode direction override characters.
- **Hash registry lookup**: checks bundle hash against the Skill Doctor community threat database. If the exact skill was previously flagged, return cached findings instantly.

Layer 3 — LLM Semantic Pass (optional, ~3–8 seconds)

- **Provider**: Groq API (Llama 3.3 70B, ~700 tokens/second). Fallback: OpenAI GPT-4o-mini. Local fallback: Ollama (llama3.2, any model).
- **Inputs**: full skill bundle text, static findings from Layer 2, declared tool schema.
- **Analysis**: semantic consistency (does the skill do what it claims?), hidden instruction extraction, tool scope audit, cross-section consistency (SKILL.md vs companion script intent).
- **Output**: structured JSON findings with `severity`, `category`, `location`, `explanation`, `remediation`.

Layer 4 — Behavioral Sandbox (optional, ~15–60 seconds)

- **Provider**: `microsandbox` by superradcompany (Integrated natively via Rust SDK).
- **Execution**: dry-run the skill in a sandboxed agent environment. The sandbox provides a mock agent runtime that loads and "uses" the skill.
- **Monitoring**: file system reads/writes, network calls (host, port, payload), environment variable access, subprocess launches, output size.
- **Comparison**: observed behavior vs. declared behavior. Divergence = finding.
- **Catches**: logic bombs, deferred payload fetches, runtime-only exfiltration paths that are invisible to static analysis.

Layer 5 — Risk Scoring and Reporting

Findings from all layers are merged, deduplicated, and scored.

Risk levels: `CRITICAL` (score ≥ 9.0), `HIGH` (7.0–8.9), `MEDIUM` (4.0–6.9), `LOW` (1.0–3.9), `INFO` (< 1.0)

Scoring factors: confidence of detection, exploitability, blast radius (how much damage if exploited), layers that confirmed the finding (cross-layer corroboration raises score).

Output formats: SARIF 2.1 (GitHub Code Scanning), JSON (structured findings), HTML (human-readable report), PDF (enterprise compliance).

Remediation: each finding includes a specific remediation suggestion and, where applicable, a patched replacement for the flagged code.

---

5. Evaluation

Note: Skill Doctor is pre-launch at time of writing. This section describes the planned evaluation protocol.

Test corpus: 1,000 skill files sampled from ClawHub, Skills.sh, and MCPServers.org. 50 synthetic malicious skills constructed to cover each SDTM-v1 attack class. 20 "evasion" skills designed to defeat static-only detection (activates only in behavioral sandbox).

Baselines: Cisco Skill Scanner (CLI, default settings), NVIDIA SkillSpector (CLI, default settings + LLM pass).

Metrics: True Positive Rate (finding real threats), False Positive Rate (flagging clean skills), Time to Scan (P50, P99), Evasion Detection Rate (Layer 4 only).

Hypothesis: Skill Doctor's multi-layer approach will achieve higher TPR than either baseline on evasion skills, with comparable FPR on clean skills, at the cost of higher P99 scan time when sandbox is enabled.

---

6. Future Work

- **Continuous monitoring**: move from point-in-time scans to runtime behavioral telemetry hooks in agent frameworks
- **Skill health history**: longitudinal security posture tracking as skills evolve across versions
- **Scientific Pack (SD-SCI)**: domain-specific rules for skills touching genomics APIs, LIMS integrations, electronic lab notebooks, and clinical trial data — the exclusive research environment rule pack
- **Registry gate API**: a pre-install scan webhook for ClawHub, Skills.sh, Lobe Hub, and MCPServers.org
- **Cryptographic provenance & Defensive Distribution**: Ed25519 skill signing and distribution via the NPM registry under the scoped namespace `@kalarislabs/skill-doctor` to thwart trademark squatting.
- **EU AI Act compliance module**: automated checks against Article 13 transparency requirements and GDPR data handling obligations

---

References

- OWASP Foundation. *Agentic Skills Top 10*. 2026.
- OWASP Foundation. *MCP Top 10*. 2026.
- NVIDIA Research. *SkillSpector: Static and Semantic Analysis of AI Agent Skill Files.* arXiv, 2026.
- Cisco AI Defense. *Skill Scanner: Open-Source Skill Security for AI Agents.* GitHub, 2026.

(Apache 2.0)
- BlueRock Security. *SSRF in the Wild: A Survey of 7,000 MCP Servers.* 2026.
- Alice.io. *Caterpillar: Scanning the Top 50 Skills on Skills.sh.* 2026.
- CISA / NSA. *Security Guidance for AI Agents in Enterprise Environments.* 2026.
- Snyk Labs. *ASCII Smuggling: Hidden Instructions in AI Skill Files.* 2026.